

实验五 —— 入侵检测实验

	姓名	学号	班级	分工	贡献
组长	项 枫	2022211570	2022211801	1. 完成实验要求的第 2 步，即实现对 Nmap 扫描的检测 2. 复现 heartbleed 漏洞，达到了对应的攻击效果，并配置实现了对该漏洞攻击的检测	50%
组员	陈璟耀	2022211685	2022211801	1. 完成实验第 1 步与第 2 步要求中的 Snort 相关配置 2. 负责撰写实验报告	50%

一、实验内容

- 1、在 linux 平台建立基于 Snort 的 IDS，能过成功运行。
- 2、在部署 Snort 的本机建立规则，实现对 Nmap 扫描的检测。
- 3、漏洞攻击检测实验：构造具有漏洞的目标环境（系统漏洞、软件漏洞、数据库漏洞、服务漏洞等），根据目标环境漏洞设计针对性攻击，达到攻击效果，并配置 snort 实现对漏洞攻击的检测。（heartbleed 漏洞攻击）

二、实验环境

虚拟机：ubuntu, kali

工具：nmap、snort

三、实验步骤

(一) 在 Linux 虚拟机部署配置 snort

1、测试是否安装成功

```
snort -V
```

```
admin@admin:~$ snort -V
_*> Snort! <*-
o"  )~
'   '
    Version 2.9.7.0 GRE (Build 149)
    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
    Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
    Copyright (C) 1998-2013 Sourcefire, Inc., et al.
    Using libpcap version 1.9.1 (with TPACKET_V3)
    Using PCRE version: 8.39 2016-06-14
    Using ZLIB version: 1.2.11
```

2、简单测试配置是否正确

```
sudo snort -T -c /etc/snort/snort.conf -I eth0
```

配置正确，启动 snort 并显示如下：

```
admin@admin:~$ sudo snort -T -c /etc/snort/snort.conf -I eth0
[sudo] admin 的密码:
Running in Test mode

--== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
/etc/snort/snort.conf(52) Var 'HOME_NET' redefined.
PortVar 'HTTP_PORTS' defined : [ 80:81 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2
809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779 8000 8008 8014 80
28 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000 9060 9080 909
0:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined : [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060:5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80:81 110 143 311 383 591 593 901 1220 1414 1741 183
0 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779 8000
8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000
```

```
--== Initialization Complete ==--

o"~)~
'""

-*> Snort! <*-
Version 2.9.7.0 GRE (Build 149)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.9.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 2.4 <Build 1>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>

Snort successfully validated the configuration!
Snort exiting
admin@admin:~$
```

(二) Snort 检测 Nmap 扫描

1、设置规则

(1) 进入 Snort/rules 目录下的 local.rules 文件中：

```
sudo vi /etc/snort/rules/local.rules
```

设置如下规则：

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Nmap Null PortScan!"; flags:0;
threshold:type both,track by_dst,count 20,seconds 60; sid:10000012; rev:1;)
```

```
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# -----
# LOCAL RULES
# -----
# This file intentionally does not come with signatures. Put your local
# additions here.
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Nmap Null PortScan!"; flags:0; threshold:
d:type both,track by_dst,count 20,seconds 60; sid:10000012; rev:1;)
~
~
~
```

(2) 注意要注释掉自带对应规则库。

打开 snort.conf

```
sudo vi /etc/snort/snort.conf
```

注释掉 include \$RULE_PATH/icmp-info.rules (608 行)

```
include $RULE_PATH/finger.rules
include $RULE_PATH/ftp.rules
# include $RULE_PATH/icmp-info.rules
include $RULE_PATH/icmp.rules
include $RULE_PATH/imap.rules
```

2、启动 snort

```
sudo snort -c /etc/snort/snort.conf -A fast -l /var/log/snort -i ens33
```

```
admin@admin:~$ sudo snort -c /etc/snort/snort.conf -A fast -l /var/log/snort -i ens33
Running in IDS mode

--== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
/etc/snort/snort.conf(52) Var 'HOME_NET' redefined.
PortVar 'HTTP_PORTS' defined : [ 80:81 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2
809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779 8000 8008 8014 80
28 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000 9060 9080 909
0:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined : [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060:5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80:81 110 143 311 383 591 593 901 1220 1414 1741 183
0 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779 8000
8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000
9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'GTP_PORTS' defined : [ 2123 2152 3386 ]
```

```
--== Initialization Complete ==--

o"~)~
'-'
'-'

-*> Snort! <*-
Version 2.9.7.0 GRE (Build 149)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.9.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 2.4 <Build 1>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Commencing packet processing (pid=2655)
```

3、查看主机 Ubuntu 的 IP 地址

```
ifconfig -a
```

```
admin@admin:~$ ifconfig -a
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.80.158 netmask 255.255.255.0 broadcast 192.168.80.255
    inet6 fe80::7938:d4fd:81f8:e0f8 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:cb:2b:84 txqueuelen 1000 (以太网)
    RX packets 410 bytes 49643 (49.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 247 bytes 29074 (29.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

4、使用kali 中的 nmap 对主机进行 NULL 扫描。nmap NULL 扫描是 nmap 秘密扫描的一种，相对比较隐蔽，通过向目标主机端口发送标志位 flags 为 0 的包进行扫描。

```
sudo nmap -sN 192.168.80.158
```

```
(kali@kali)~$ sudo nmap -sN 192.168.80.158
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-12 07:08 EST
Nmap scan report for 192.168.80.158
Host is up (0.0014s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    open|filtered ssh
MAC Address: 00:0C:29:CB:2B:84 (VMware)
Nmap done: 1 IP address (1 host up) scanned in 1.47 seconds
```

5、在日志中查看检测报告

```
cat /var/log/snort/alert
```

```
admin@admin:~$ cat /var/log/snort/alert
12/12-19:59:14.159799  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.80.1:58608 -> 239.255.255.250:1900
12/12-19:59:26.623001  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {UDP} 0.0.0.0:68 -> 255.255.255.255:67
12/12-19:59:26.634722  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {IPV6-ICMP} :: -> ff02::16
12/12-19:59:27.212000  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {IPV6-ICMP} :: -> ff02::1:fff8:e0f8
12/12-20:04:38.663002  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {IPV6-ICMP} :: -> ff02::1:fff8:e0f8
12/12-20:08:57.346611  [**] [1:10000012:1] Nmap Null PortScan! [**] [Priority: 0] {TCP} 192.168.80.157:57272 -> 192.168.80.158:443
12/12-20:08:57.372587  [**] [1:1421:11] SNMP AgentX/tcp request [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.80.157:57272 -> 192.168.80.158:705
12/12-20:08:57.412571  [**] [1:1418:11] SNMP request tcp [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.80.157:57272 -> 192.168.80.158:161
12/12-20:09:34.299541  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.80.1:61552 -> 239.255.255.250:1900
```

(三) Heartbleed 漏洞攻击检测实验

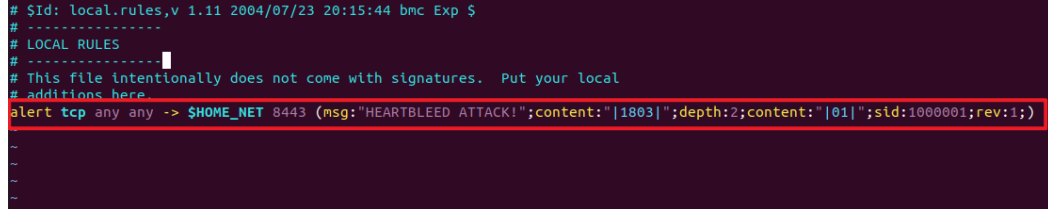
1、设置规则

(1) 进入 Snort/rules 目录下的 local.rules 文件中：

```
sudo vi /etc/snort/rules/local.rules
```

设置如下规则：

```
alert tcp any any -> $HOME_NET 8443 (msg:"HEARTBLEED  
ATTACK!";content:"|1803|";depth:2;content:"|01|";sid:1000001;rev:1;)
```



```
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $  
# -----  
# LOCAL RULES  
# -----  
# This file intentionally does not come with signatures. Put your local  
# additions here.  
alert tcp any any -> $HOME_NET 8443 (msg:"HEARTBLEED ATTACK!";content:"|1803|";depth:2;content:"|01|";sid:1000001;rev:1;)  
~  
~  
~
```

规则解释：心脏出血漏洞主要通过攻击者模拟向服务器端发送自己编写的 Heartbeat 心跳数据包，主要是 HeartbeatMessage 的长度与 payload 的 length 进行匹配，若 payload_length 长度大于 HeartbeatMessage 的 length，则会在服务器返回的 response 响应包中产生数据溢出，造成有用数据泄露。

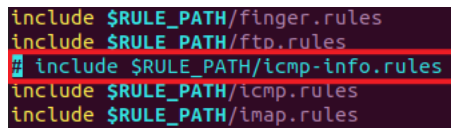
HeartbeatMessage 的 length 为 2^{14} 字节即 16384 字节，协议是 TCP，面向的端口是 443 即 HTTP/HTTPS 服务,本实验采用 docker 环境复现,因此端口使用 8443。

(2) 注意要注释掉自带对应规则库。

打开 snort.conf

```
sudo vi /etc/snort/snort.conf
```

注释掉 include \$RULE_PATH/icmp-info.rules (608 行)



```
include $RULE_PATH/finger.rules  
include $RULE_PATH/ftp.rules  
# include $RULE_PATH/icmp-info.rules  
include $RULE_PATH/icmp.rules  
include $RULE_PATH/imap.rules
```

2、启动 snort

```
sudo snort -c /etc/snort/snort.conf -A fast -l /var/log/snort -i ens33
```

```
admin@admin:~$ sudo snort -c /etc/snort/snort.conf -A fast -l /var/log/snort -i ens33
Running in IDS mode

--== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
/etc/snort/snort.conf(52) Var 'HOME_NET' redefined.
PortVar 'HTTP_PORTS' defined : [ 80:81 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2
809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779 8000 8008 8014 80
28 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000 9060 9080 909
0:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined : [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060:5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80:81 110 143 311 383 591 593 901 1220 1414 1741 183
0 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779 8000
8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000
9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'GTP_PORTS' defined : [ 2123 2152 3386 ]
```

```
[ Number of patterns truncated to 20 bytes: 1038 ]
pcap DAQ configured to passive.
Acquiring network traffic from "ens33".
Reload thread starting...
Reload thread started, thread 0x7f7db8cc8700 (2660)
Decoding Ethernet

--== Initialization Complete ==--

--> Snort! <*-
o" )~ Version 2.9.7.0 GRE (Build 149)
' ' By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.9.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 2.4 <Build 1>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Commencing packet processing (pid=2655)
```

3、启动 docker。

```
sudo systemctl start docker
```

```
admin@admin:~$ sudo systemctl start docker
admin@admin:~$
```

4、进入环境所在目录启动环境。

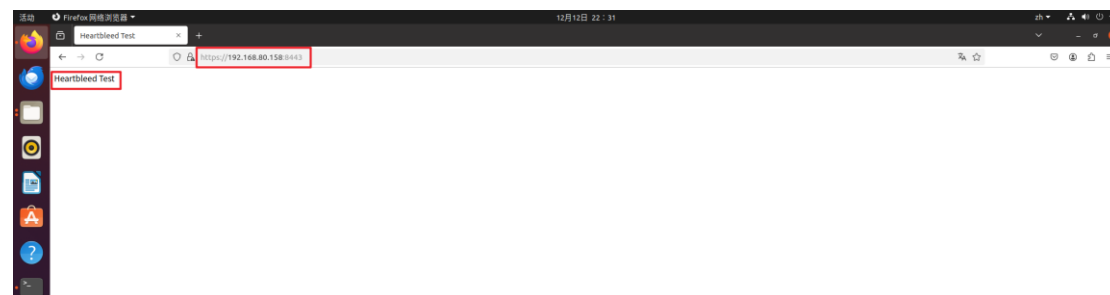
```
cd /home/admin/桌面/vulhub-master/openssl/CVE-2014-0160/
sudo docker-compose up -d
```

```
admin@admin:~/桌面/vulhub-master/openssl/CVE-2014-0160$ cd /home/admin/桌面/vulhub-master/openssl/CVE-2014-0160/
admin@admin:~/桌面/vulhub-master/openssl/CVE-2014-0160$ sudo docker-compose up -d
Pulling nginx (vulhub/openssl:1.0.1c-with-nginx)...
1.0.1c-with-nginx: Pulling from vulhub/openssl
b281ebec60d2: Pull complete
2700c1ade95c: Pull complete
cd1f945398e5: Pull complete
24291727d0f3: Pull complete
c661453e1eb5: Pull complete
f4f1857f7bb1: Pull complete
951d0b01db0f: Pull complete
Digest: sha256:3cf76769b6e33f45479e8bacd70d7c5c2bd8b8c4d5428ae3f613b1145a4a1c47
Status: Downloaded newer image for vulhub/openssl:1.0.1c-with-nginx
Creating cve-2014-0160_nginx_1 ... done
admin@admin:~/桌面/vulhub-master/openssl/CVE-2014-0160$
```

```
sudo docker ps
```

```
admin@admin:~/桌面/vulhub-master/openssl/CVE-2014-0160$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  NAMES                  CREATED        STATUS        PORTS
66e41ac7acbc   vulhub/openssl:1.0.1c-with-nginx   "/usr/local/nginx/sbin" 33 seconds ago   Up 31 seconds   0.0.0.0:8080->80/tcp,
[::]:8080->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp   cve-2014-0160_nginx_1
admin@admin:~/桌面/vulhub-master/openssl/CVE-2014-0160$
```

5、打开浏览器输入 <https://192.168.80.158:8443>，出现 Heartbleed Test 说明环境启动成功。



6、使用 msf 利用漏洞。

```
msfconsole
```

```
(kali@kali)~$ msfconsole
Metasploit tip: Start commands with a space to avoid saving them to history

      .:ok000kdc'      'cdk000ko:
      .x000000000000c      c00000000000x
      :00000000000000k,      ,k000000000000000
      '00000000kkk00000: :000000000000000000'
      00000000, ,0000000000l, ,000000000
      d0000000, ,c00000c, ,00000000x
      l0000000, ;d; ,00000000l
      ,0000000, ;; ,00000000,
      c0000000, .00c, '000, ,0000000c
      0000000, .0000, :0000, ,0000000
      l00000, .0000, :0000, ,000000l
      ;0000' .0000: :0000: :0000;
      .d000 ,0000cccx0000, x00d,
      ,kol ,000000000000, .00k,
      ,kk; ,000000000000, c0k;
      ;k000000000000000k;
      ,x000000000000x,
      ,l0000000l,
      ,d0d,
      .

+ -- ==[ 2437 exploits - 1255 auxiliary - 429 post ]
+ -- ==[ 1468 payloads - 47 encoders - 11 nops ]
+ -- ==[ 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/

msf6 >
```

7、打开 msf 后，查找 heartbleed 模块，查看需要设置哪些选项。

```
search heartbleed
use auxiliary/scanner/ssl/openssl_heartbleed
show options
```

```
msf6 > search heartbleed
Matching Modules
#  Name
0  auxiliary/scanner/http/elasticsearch_memory_disclosure
1  \_ action: DUMP
2  \_ action: SCAN
3  auxiliary/server/openssl_heartbeat_client_memory
4  auxiliary/scanner/ssl/openssl_heartbleed
5  \_ action: DUMP
6  \_ action: KEYS
7  \_ action: SCAN
Disclosure Date  Rank  Check  Description
2021-07-21      normal Yes  Elasticsearch Memory Disclosure
.              .      .      Dump memory contents to loot
.              .      .      Check hosts for vulnerability
2014-04-07      normal No   OpenSSL Heartbeat (Heartbleed) Client Memory Exp
2014-04-07      normal Yes  OpenSSL Heartbeat (Heartbleed) Information Leak
.              .      .      Dump memory contents to loot
.              .      .      Recover private keys from memory
.              .      .      Check hosts for vulnerability

Interact with a module by name or index. For example info 7, use 7 or use auxiliary/scanner/ssl/openssl_heartbleed
After interacting with a module you can manually set a ACTION with set ACTION 'SCAN'

msf6 > use auxiliary/scanner/ssl/openssl_heartbleed
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > show options

Module options (auxiliary/scanner/ssl/openssl_heartbleed):
Name          Current Setting  Required  Description
DUMPFILTER    1               no        Pattern to filter leaked memory before storing
LEAK_COUNT    1               yes       Number of times to leak memory per SCAN or DUMP invocation
MAX_KEYTRIES  50              yes       Max tries to dump key
RESPONSE_TIMEOUT 10             yes       Number of seconds to wait for a server response
RHOSTS        .               yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT         443             yes       The target port (TCP)
STATUS_EVERY  5               yes       How many retries until key dump status
THREADS       1               yes       The number of concurrent threads (max one per host)
TLS_CALLBACK  None            yes       Protocol to use, "None" to use raw TLS sockets (Accepted: None, SMTP, IMAP, JABBER, POP3, FTP, POSTGRES)
TLS_VERSION   1.0             yes       TLS/SSL version to use (Accepted: SSLv3, 1.0, 1.1, 1.2)

Auxiliary action:
Name  Description
SCAN  Check hosts for vulnerability

View the full module info with the info, or info -d command.
msf6 auxiliary(scanner/ssl/openssl_heartbleed) >
```

8、设置靶机地址 RHOSTS 和靶机端口号 RPORT，同时设置 verbose 以看到泄露的信息。

```
set RHOSTS 192.168.80.158
set RPORT 8443
set verbose true
```

```
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set RHOSTS 192.168.80.158
RHOSTS => 192.168.80.158
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set RPORT 8443
RPORT => 8443
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set verbose true
verbose => true
msf6 auxiliary(scanner/ssl/openssl_heartbleed) >
```

9、exploit，可以看到泄露的数据，漏洞利用成功。

```
exploit
```


10、在日志中查看检测报告

```
cat /var/log/snort/alert
```

```
admin@admin:~$ cat /var/log/snort/alert
12/12-19:59:14.159799  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.80.1:58608 -> 239.255.255.250:1900
12/12-19:59:26.623001  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {UDP} 0.0.0.0:68 -> 255.255.255.255:67
12/12-19:59:26.634722  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {IPv6 -ICMP} :: -> ff02::16

12/12-20:25:17.572131  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {IPv6 -ICMP} :: -> ff02::1:ff1e:e22f
12/12-20:30:14.439187  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.80.1:50422 -> 239.255.255.250:1900
12/12-20:30:20.122418  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {UDP} 0.0.0.0:68 -> 255.255.255.255:67
12/12-20:30:26.181395  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {UDP} 0.0.0.0:68 -> 255.255.255.255:67
12/12-20:30:27.420738  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {UDP} 0.0.0.0:68 -> 255.255.255.255:67
12/12-20:30:27.434422  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {IPv6 -ICMP} :: -> ff02::16
12/12-20:30:27.578996  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {IPv6 -ICMP} :: -> ff02::16
12/12-20:30:28.138815  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {IPv6 -ICMP} :: -> ff02::1:ff1e:e22f
12/12-20:35:11.752676  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {IPv6 -ICMP} :: -> ff02::16
12/12-21:22:34.835716  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {IPv6 -ICMP} :: -> ff02::16
12/12-21:22:34.875448  [**] [1:527:8] BAD-TRAFFIC same SRC/DST [**] [Classification: Potentially Bad Traffic] [Priority: 2] {ICMP} 0.0.0.0 -> 224.0.0.22
12/12-23:13:59.131130  [**] [1:1000001:1] HEARTBLEED ATTACK! [**] [Priority: 0] {TCP} 192.168.80.157:44595 -> 192.168.80.158:8443
admin@admin:~$
```

四、总结

通过本次实验，我们成功完成了基于 Snort 的入侵检测系统（IDS）的搭建与规则配置，并验证了其对网络扫描和漏洞攻击的检测能力。

首先，我们在 Linux 环境中部署了 Snort，并通过自定义规则实现了对 Nmap NULL 扫描的准确检测，规则的设置体现了对标志位和阈值的合理运用，有效捕获了隐蔽的扫描行为。随后，在漏洞攻击检测实验中，我们复现了 Heartbleed 漏洞攻击，通过自定义 Snort 规则，从攻击端口和特定数据包内容等特征入手，实现了对漏洞攻击的实时检测。本次实验不仅强化了我们入侵检测系统工作原理的理解，还提升了我们在网络安全工具使用和规则设计方面的实践能力。尤其是在规则配置过程中，深刻体会到准确理解网络行为特征的重要性。与此同时，实验中遇到的诸如环境搭建、工具配置等问题也锻炼了我们的排查和解决问题的能力。

总之，此次实验加深了我们对网络攻防技术的认识，为后续深入研究和应用安全技术奠定了坚实基础。